## Algorithm

An algorithm is a step-by-step procedure or formula for solving a problem. It is a well-defined sequence of instructions that, when followed, solve a particular task or problem.

### Conditions

- ☐ Finiteness → It must have a finite number of steps. It should not run indefinitely.
- ☐ Definiteness → Each step must be precisely defined, with no ambiguity in the instructions.
- ☐ Input → It should accept zero or more inputs.
- ☐ Output → It should produce at least one output, representing the solution to the problem.
- ☐ Effectiveness → Every step of the algorithm must be basic enough to be carried out, in principle, by a person or machine.

# Recurrence Relation of Algorithm

A recurrence relation is a mathematical expression that defines a sequence in terms of its previous terms. It is often used to model the time complexity of recursive algorithms.

## General form:

$$a_n = f(a_{n-1}, a_{n-2}, \ldots a_{n-k})$$

where f is a function that defines the relationship between the current term and the previous terms.

## Common uses:

- ☐ Time complexity analysis
- ☐ Generalizing Divide and Conquer Algorithms
- ☐ ~~Anala~~ Analyzing Recursive Algorithms
- ☐ Defining State and Transitions for Dynamic Programming

## Examples:

- **Binary Search:**

  We work on one half of the array at once.

```
BS (arr, l, h, key){
  if (l > h) return -1;
  m = (l+h)/2;
  if (arr[m] > key)  BS (arr, l, m-1, key);
  else if (arr[m] < key)  BS (arr, m+1, h, key);
  else return ~~mid;~~ m;
}
```

$T(n) = T(n/2) + const.$
↓
for comparison

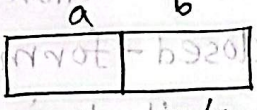
- ~~Fab~~ Fibonacci Sequence:

```
fib(n) {
    if (n == 0 || n == 1)
        return ~~0~~ n;
    return fib(n-1) + fib(n-2);
}
```

$T(n) = T(n-1) + T(n-2) + const.$
↓ Time to solve first sub-problem
↓ Time to solve second sub-problem
↳ Time for the 'if' check and the '+' addition

- Merge Sort:

Merge sort works by splitting an array of size n into two halves, sorting each ~~half~~ half recursively, and then merging them back together.




$T(n) = T(n/2) + T(n/2) + cn$ → [a constant $\leq 0 < c \leq 1$]
↓ Sort left half
↓ Sort right half
↓ Time to merge → can be less than n total operations.
$= 2T(n/2) + cn$

## Ways to solve Recurrence Relations:

(i) Substitution Method (Iteration)
(ii) Recursive Tree Method
(iii) Master Method

## Substitution Method

The substitution method solves a recurrence relation by repeatedly replacing the recurrence with itself until a pattern appears, then expressing the result in closed form.

### Steps:

- ☐ Write the given recurrence relation and identify the base (initial) condition.
- ☐ Expand (substitute) the recurrence repeatedly by replacing each recursive term with its recurrence.
- ☐ Observe and write general pattern after k substitutions.
- ☐ Determine the value of k by using the base condition (make the recursive term equal to the base case).
- ☐ Substitute the value of k into the general pattern.
- ☐ Simplify the expression (use summation formulas if necessary) to obtain the closed-form solution.
- ☐ Verify the result by checking that it satisfies the initial condition.

[Write → Expand → Pattern → Find k → Substitute
↓
Verify ← Simplify]

# $T(n) = T(n/2) + c$

$T(n/2) = T(n/4) + c = T(n/2^2) + c \Rightarrow T(n) = T(n/2^2) + 2c$

$T(n/4) = T(n/8) + c = T(n/2^3) + c \Rightarrow T(n) = T(n/2^3) + 3c$

$$T(n) = \begin{cases} T(n/2) + c & [n > 1] \\ 1 & [n == 1] \end{cases}$$

~~$T\left(\frac{n}{2^{k-1}}\right)$~~

$\therefore T(n) = T\left(\frac{n}{2^k}\right) + k \cdot c$

Now, $\frac{n}{2^k} = 1$

$\Rightarrow n = 2^k$

$\Rightarrow k = \log_2 n$

$T(n) = T(1) + c\log_2 n \approx c \cdot \log_2 n \rightarrow O(\log n)$

# Merge Sort:

$T(n) = 2T(n/2) + n$

$= 2\{2T(n/2^2) + n/2\} + n$

$= 2^2 T\left(\frac{n}{2^2}\right) + 2n$

$= 2^2\left\{2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right\} + 2n$

$= 2^3 T\left(\frac{n}{2^3}\right) + 3n$

- - - - - - -

- - - - - - -

$= 2^k T\left(\frac{n}{2^k}\right) + k \cdot n$

CamScanner

Base case boundary:

$$\frac{n}{2^k} = 1$$

$$\Rightarrow n = 2^k$$

$$\Rightarrow k = \log_2 n$$

When $n == 1$,
$T(n) = 1$

$$T(n) = 2^{\log_2 n} \cdot T(1) + n\log_2 n$$
$$= n \cdot 1 + n\log_2 n$$
$$= n(1 + \log_2 n)$$
$$\approx n\log_2 n \rightarrow O(n\log n)$$

Complexity order:
$c < \log n < n < n\log n < e^n < n^n < n!$

# $T(n) = nT(n-1) + \log_2 n$

$$= n\{(n-1)T(n-2) + \log_2(n-1)\} + \log_2 n$$
$$= n(n-1)T(n-2) + n\log_2(n-1) + \log_2 n$$
$$= n\{n-2$$
$$= n$$

$[n(n-1) \approx n^2]$

$$\approx n^2\{(n-2)T(n-3) + \log_2(n-2)\} + n\log_2(n-1)^2 +$$
$$n\log_2(n-1) + \log_2 n$$
$$= n^2(n-2)T(n-3) + n^2\log_2(n-2) + n\log_2(n-1) + n^0\log_2 n$$
$$\approx n^3 T(n-3) + n^2\log_2(n-2) + n^1\log_2(n-1) + n^0\log_2(n-0)$$

$[n^2(n-2) \approx n^3]$

$$= n^k T(n-k) + n^{k-1}\log(n-(k-1)) + n^{k-2}\log_2(n-(k-2)) +$$
$$\cdots + n^{k-k}\log_2(n-(k-k))$$

Base case boundary:

$n-k=1$

$\Rightarrow n-1=k$

$\Rightarrow k=n-1$

$2^k = 1^k - 1$

Now, $T(n) = T(1) + \log(n-(n-1-1)) + \dots + \log n$

$= 1 + \log 2 + \log 3 + \dots + \log n$

$= 1 + \log(2.3.\dots.n)$

$= 1 + \log(n!)$

$\approx \log(n!)$

Best case $\Rightarrow 1+1=2 \Rightarrow O(1)$ [constant time]

Worst case $\Rightarrow 1 + n\log n \Rightarrow O(n\log n)$ [$\log(n!) \approx n\log n - n$ and $n\log n > n$]

$$\# T(n) = \begin{cases} 2T(n-1)-1 & ; n>0 \\ 1 & ; n=0 \text{ or } n=1 \end{cases}$$

$T(n) = 2T(n-1) - 1$

$= 2\{2T(n-2)-1\} - 1$

$= 2^2 T(n-2) - 2 - 1$

$= 2^2(2T(n-3)-1) - 2 - 1$

$= 2^3 T(n-3) - 2^2 - 2^1 - 2^0$

- - - - -

- - - - -

$= 2^k T(n-k) - 2^{k-1} - 2^{k-2} - \dots - 2^2 - 2^1 - 2^0$

Base case boundary:

$n-k=1$

$\Rightarrow k=n-1$

Now,

$$T(n) = 2^{n-1}\cdot T(1) - 2^{n-2} - 2^{n-3} - \dots - 2^2 - 2^1 - 2^0$$

$$= 2^{n-1} - 2^{n-2} - 2^{n-3} - \dots - 2^2 - 2^1 - 2^0$$

$$= 2^{n-1} - (2^{n-2} + 2^{n-3} + \dots + 2^2 + 2^1 + 2^0)$$

$$= 2^{n-1} - \sum_{i=0}^{n-2} 2^i$$

$$= 2^{n-1} - 2^0\left(\frac{2^{n-1}-1}{2-1}\right) \qquad \left[S_m = a\left(\frac{r^m-1}{r-1}\right)\right]$$

(a → first term, r → ratio)

$$= 2^{n-1} - (2^{n-1} - 1)$$

$$= 1$$

$$= \cancel{O(n}\ O(1)$$

↳ Constant time

## Recursion Tree Method

Structure:

$$T(n) = T(\text{...}) + \text{cost}$$

- $T(\text{...})$ ↓ recursive form
- cost ↓ additional cost to solve subproblems.
  - ↳ the non-recursive work done at the current level.

~~# Merge Sort~~

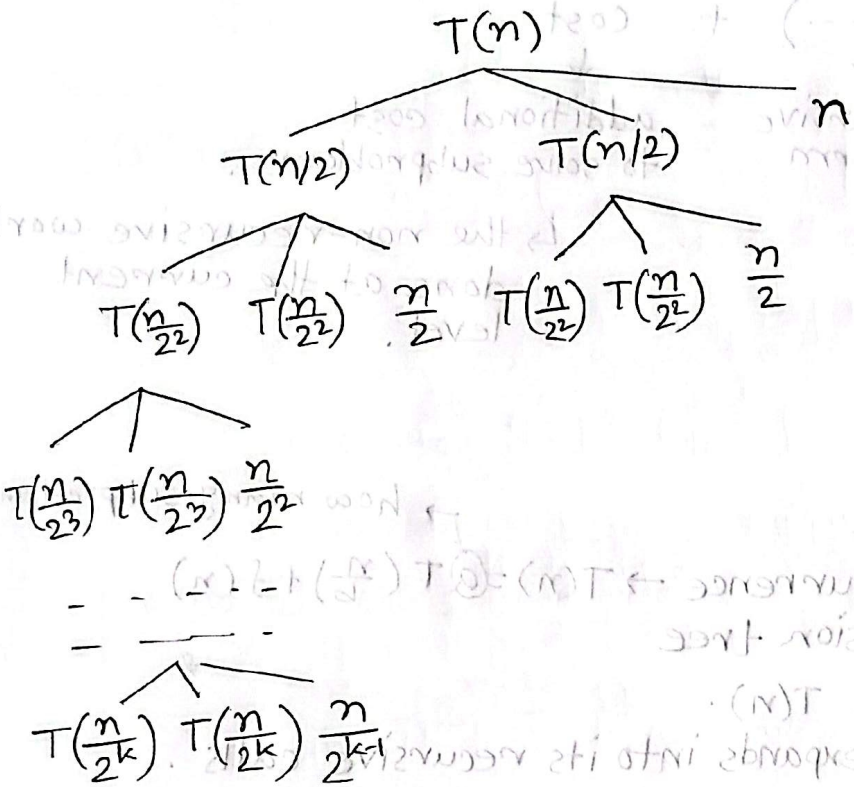
Steps:

- ☐ Write the recurrence → $T(n) = @T(\frac{n}{b}) + f(n)$
  - @ ↑ how many subproblems?
- ☐ Draw the recursion tree
  - The root is $T(n)$.
  - Each node expands into its recursive calls.
- ☐ Find the cost at each node.
  - ↳ non-recursive work $f$(subproblem size)
- ☐ Find the ~~nut~~ number of nodes at each level
  - Level $i$ typically has @$^i$ nodes
    - @ ↳ ~~sub-tree~~ number of recursive calls made by each function call (number of subproblems)
- ☐ Find the cost of each level:
  - Level cost = number of nodes * cost per node
- ☐ Determine the height of the tree (usually $\log_b n$)
- ☐ Find leaf cost (~~last~~ node found at $k$ level)
- ☐ Sum all level costs and simplify

CamScanner

# Merge Sort :

$$T(n) = 2T(n/2) + n$$




$n$

$\frac{n}{2} \times 2 = n$

$\frac{n}{2^2} \times 4 = n$

$n$

Steps : $h.n$ [h = height of tree]

Base case boundary :

$$T\left(\frac{n}{2^k}\right) = T(1)$$

$$\Rightarrow \frac{n}{2^k} = 1$$

$$\Rightarrow n = 2^k$$

$$\Rightarrow k = \log_2 n$$

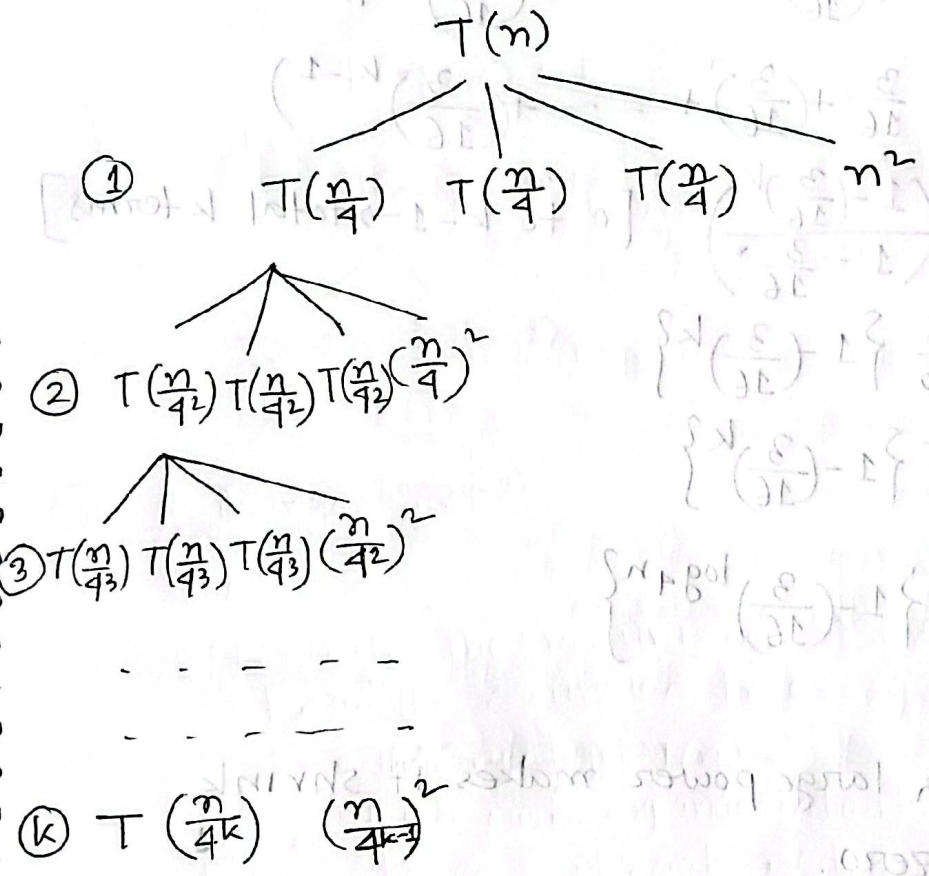
Since, the tree goes down to k levels, the height of the tree is,

$$h = \log_2 n$$

$\therefore T(n)$ = height × work per level

$$= \boxed{n \log n}$$

# $T(n) = 3T(n/4) + n^2$




$n^2 = \left(\frac{3}{16}\right)^0 n^2$

$\frac{n^2}{4^2} \times 3 = \frac{3n^2}{4^2} = \left(\frac{3}{16}\right)^1 n^2$

$\left(\frac{n^2}{4^2}\right)^2 \times 9 = \frac{3^2}{16^2} n^2$

$\left(\frac{n}{4^{k-1}}\right)^2 \times 3^{k-1} = \left(\frac{3}{16}\right)^{k-1} n^2$

Here, $\frac{3}{16} < 1$. This means that as we go deeper down into the tree, the amount of work at each level gets smaller and smaller.

Now, $\frac{n}{4^k} = 1$

$\Rightarrow n = 4^k$

$\Rightarrow \log_4 n = k$

$\Rightarrow k = \log_4 n$

CamScanner

Thus, total height, $h = \log_4 n$

So, $T(n) = n^2 + \frac{3}{16} n^2 + \left(\frac{3}{16}\right)^2 n^2 + \cdots + \left(\frac{3}{16}\right)^{k-1} n^2$

$= n^2 * \left(1 + \frac{3}{16} + \left(\frac{3}{16}\right)^2 + \cdots + \left(\frac{3}{16}\right)^{k-1}\right)$

$= n^2 \cdot 1.\left(\frac{1 - \left(\frac{3}{16}\right)^k}{1 - \frac{3}{16}}\right)$ [0 to $k-1$ → total $k$ terms]

$= n^2 \cdot \frac{16}{13}\left\{1 - \left(\frac{3}{16}\right)^k\right\}$

$= \frac{16}{13} n^2 \left\{1 - \left(\frac{3}{16}\right)^k\right\}$

$= \frac{16}{13} n^2 \left\{1 - \left(\frac{3}{16}\right)^{\log_4 n}\right\}$

$\because \frac{3}{16} < 1$

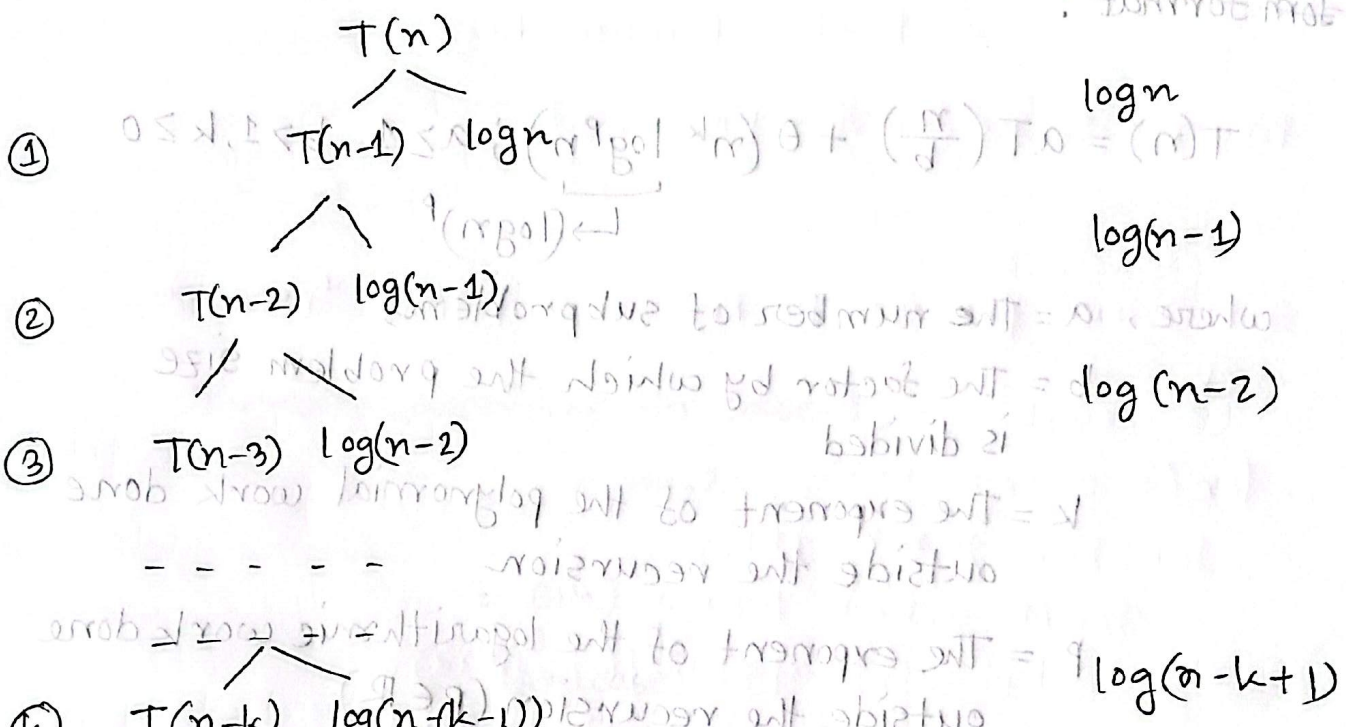
$\therefore$ Raising it to a large power makes it shrink rapidly towards zero.

So, $T(n) \approx \frac{16}{13} n^2 \{1 - 0\}$

$= \frac{16}{13} n^2$

$\approx n^2 \longrightarrow O(n^2)$

# $T(n) = \begin{cases} T(n-1) + \log n & ; n > 0 \\ 0 & ; n = 1 \end{cases}$




Base case boundary: $n - k = 0$

$\Rightarrow k = n$

Now,

$$T(n) = \log n + \log(n-1) + \log(n-2) + \cdots + \log(n-k+1)$$

$$= \log n + \log(n-1) + \log(n-2) + \cdots + \log 2 + \log 1$$

$$= \log\{n(n-1)(n-2)\cdots 2 \cdot 1\}$$

$$= \log(n!)$$

$$\approx n\log n - n \rightarrow O(n \log n)$$

## Master Method

It's used to ~~solve~~ quickly solve divide-and-conquer ~~recurrence~~ recurrence relations that fit this specific ~~form~~ format :

$$T(n) = aT\left(\frac{n}{b}\right) + \theta\left(n^k \underbrace{\log^p n}_{\hookrightarrow (\log n)^p}\right); \quad a \geq 1, b > 1, k \geq 0$$

where, $a$ = The number of subproblems

$b$ = The factor by which the problem size is divided

$k$ = The exponent of the polynomial work done outside the recursion

$p$ = The exponent of the logarithmic work done outside the recursion ($p \in \mathbb{R}$)

The three master cases :

Case-1 : $a > b^k \longrightarrow \theta(n^{\log_b a})$

Case-2 : $a = b^k \longrightarrow$
- if $p > -1$; then $\theta(n^k \log^{p+1} n)$
- if $p = -1$; then $\theta(n^k \log\log n)$
- if $p < -1$; then $\theta(n^k)$

Case-3 : $a < b^k \longrightarrow$
- if $p \geq 0$; then $\theta(n^k \log^p n)$
- if $p < 0$; then $\theta(n^k)$

CamScanner

# $T(n) = 2T(\frac{n}{2}) + 1$

Here,

$a = 2$

$b = 2$

$k = 0$

$\therefore b^k = 2^0 = 1$

$\therefore a > b^k$

Therefore, according to case-1,

$$T(n) = \theta(n^{\log_2 2})$$

$$= \theta(n)$$

↓ (Avg case)

## Brute Force Algorithm

Why to use : (i) Simplicity
(ii) Completeness
(iii) Baseline Benchmarking

Why not to use : (i) Very inefficient

# Maximum product of three element from three arrays each:

```
for (i=0; i<n; i++){
  for (j=0; j< n; j++){
    for (k=0; k<n; k++){
      if (a[i] * a[j] * a[k] >max){
        max= a[i]* a[j] * a[k];
      }
    }
  }
}
```

# Greedy Algorithm

Core properties: (i) Greedy choice property
(ii) Optimal substructure

Examples: (i) Fractional Knapsack
(ii) Job Sequencing
(iii) Dijkstra's Algorithm (Shortest Path Algorithm — SPA)
(iv) Prim's Algorithm (Minimum Spanning Tree - MST)
(v) Kruskal's Algorithm (MST)

Fractional Knapsack example:

Total knapsack capacity (W) = 17
Number of objects: 7

| Object ID | Profit | Weight(W) | P/W |
|---|---|---|---|
| 1 | 5 | 1 | 5.00 |
| 2 | 10 | 3 | 3.33 |
| 3 | 15 | 5 | 3.00 |
| 4 | 7 | 4 | 1.75 |
| 5 | 8 | 1 | 8.00 |
| 6 | 9 | 3 | 3.00 |
| 7 | 4 | 2 | 2.00 |

Max profit = ?

CamScanner

Soln:

To maximize profit, items are selected in descending order of their profit_to_weight ratio (P/W):

Choice order: 5→1→2→3→6→7→4

| Object ID | Profit | Remaining Weight |
|---|---|---|
| 5 | 8 | 17-1=16 |
| 1 | 5 | 16-1=15 |
| 2 | 10 | 15-3=12 |
| 3 | 15 | 12-5=7 |
| 6 | 9 | 7-3=4 |
| 7 | 4 | 4-2=2 |
| 4 | 7/2=3.5 | 2-2=0 |
| | 54.5 | |

↳ Maximum profit

Implementation:

```
struct item {
    int obj_ID;
    float profit;
    float weight;
    float PWR;
};item;
struct item it[7];
```

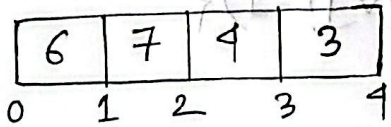
```
for (i=0; i<size; i++){
    if (~~wi~~ c==0)
        break;
    if (wi<=c){
        c = c - wi;
        p = p + pi;
    }
    else{
        p = p + pWpi * c;
        c = 0;
    }
}
```

# Job Sequencing Problem & Dijkstra's Algorithm

Total jobs = 7.
Deadline unit : months.

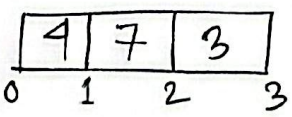
| Job ID | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| Profit | 3 | 5 | 20 | 18 | 1 | 6 | 30 |
| Deadline | 1 | 3 | 4 | 3 | 2 | 1 | 2 |

Profit order : 7 > 3 > 4 > 6 > 2 > 1 > 5




[If deadline was 4 months in total]
Max profit = 6+30+18+20 = 74 $

If deadline was 3 months in total:




Max profit = 18+30+20 = 68 $

CamScanner

Pseudocode :

```
dijkstr (G, s) {
    for each v ∈ G.V {
        dist [v] = ∞;
        par [v] = NULL;
        visited [v] = 0;}
    dist[s] = 0;
    par[s] = -1;
    PQ {s, 0}

while (PQ! =empty){
    // Remove the node with min. dist. from Priority Queue
(Mean-Hea (Min-heap)
    u,d = extractMin (PQ) //removeTop (PQ)

    // Skip if we're already processed this node
    if (visited [u] == 1){
        continue;
    }
    //Relax all unvisited neighbours
    for each neighbour v of u{
        dd = dist[u] + edge(u,v); //calculate new tentative dist.
        if(dd < dist[v] && visited [v]! =1){
            dist[v] = dd;
            par[v] = u;
            PQ.decreasekey ({v, dt}); //update dist. inside
priority queue)
}  }
```

// Mark node u as fully processed
visited[u] = 1;
}

Space complexity :

Total space = V + V + V + E + E = 3V + 2E ≈ O(V+E)

(dist. → V, par. → V, visited → V, PQ → E, Graph → E)

Time complexity:

- ExtractMin → V log V
- DecreaseKey → E log V

Total time = V log V + E log V
= O((V+E) log V)

## Divide and Conquer

Break a problem down into smaller sub-problems, Solve them recursively, and combine their results.

Example: Binary Search → divided in half at every step.

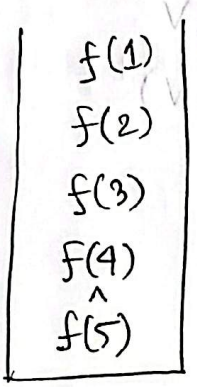
$T(n) \rightarrow O(\log n)$

## Recursive Algorithm (Stack thinking)

When an algorithm uses recursion, the system ~~stack~~ tracks active function calls using a call stack.

### Linear Recursion (Single branch)

f(5)
↓
f(4)
↓
f(3)
↓
f(2)
↓
f(1)



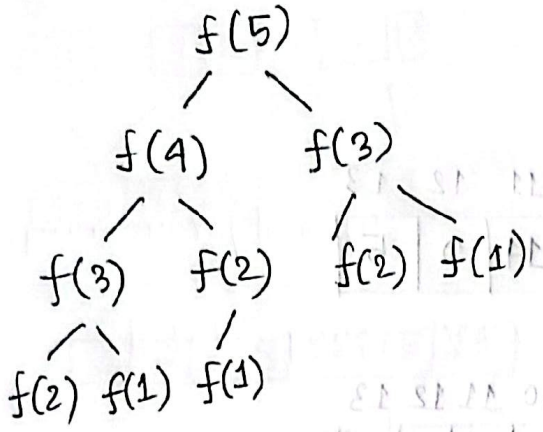

h = height of the call stack

m = memory space for each ~~ela~~ call

Total space = $h \times m \approx O(h)$

Tree-Based Recursion (Multiple branches)

Fibonacci :




Maximum depth or height = ~~n~~,

$h = n$

For recursive binary ~~tree~~ search tree,

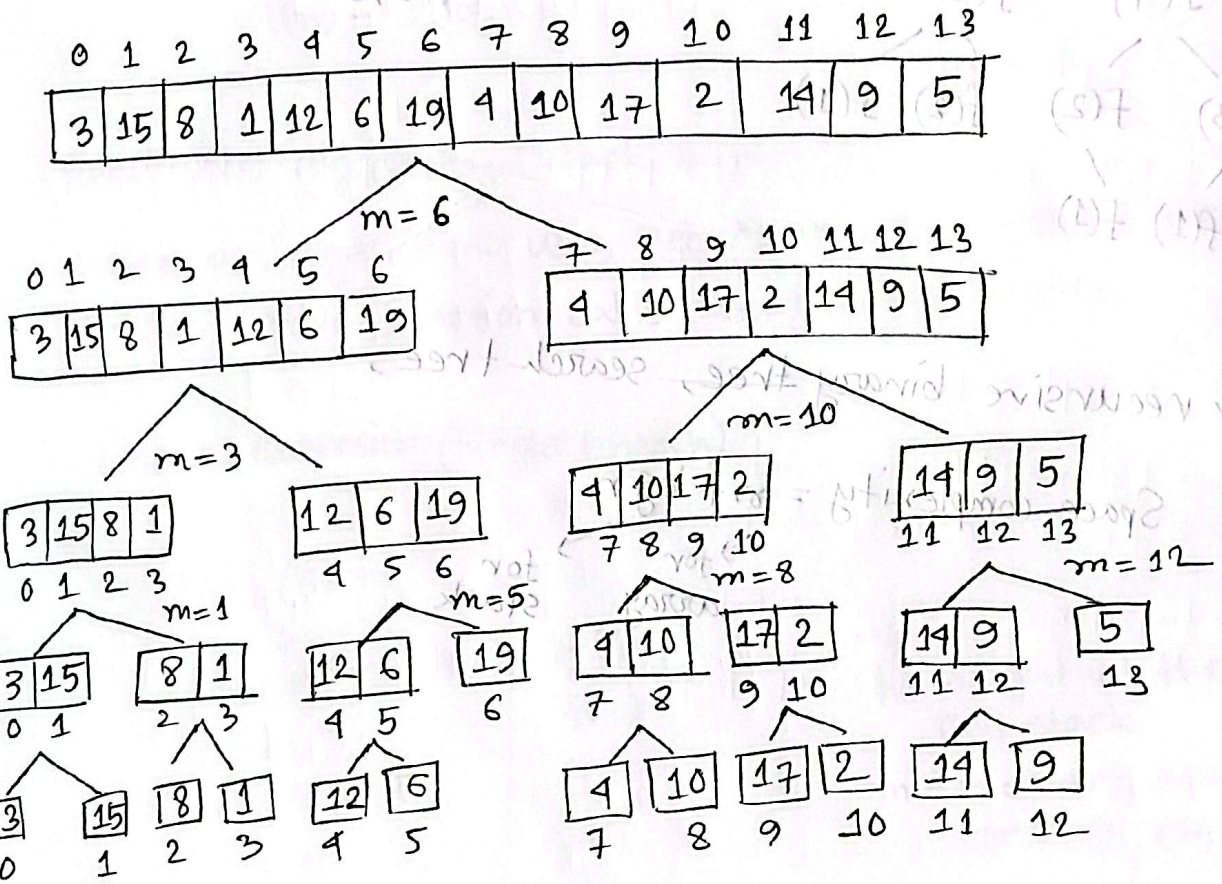
Space complexity = $n + \log n$

↘ for array ↘ for stack

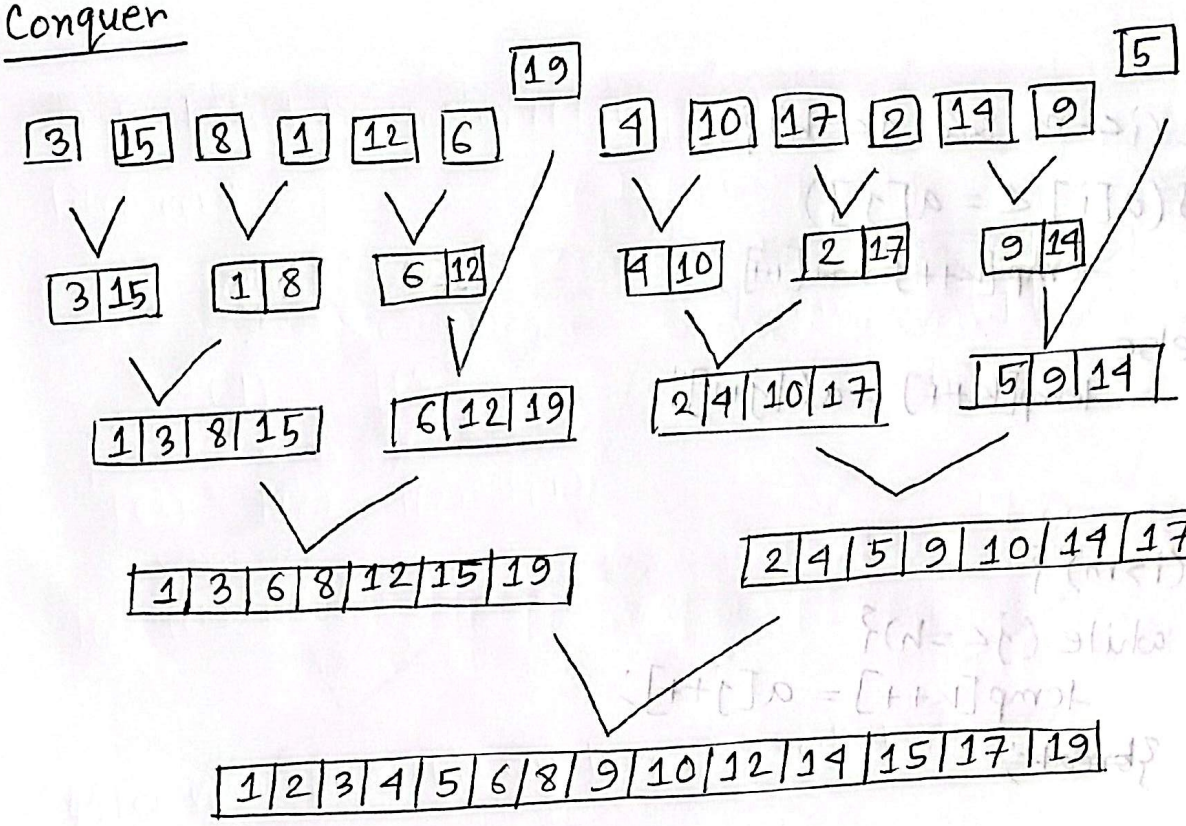
# Divide and Conquer example:

Merge Sort:

Divide



Conquer




Implementation:

```
mergesort (a[], l, h){
  if (l<h){
     m=(l+h)/2;
     mergesort(a, l, m);
     mergesort (a, m+1, h);
      combine (a, l, m, h);
  }

combine (a[], l, m, h){
   int i = l;
   j = m+1;
   k = 0;
   .
```

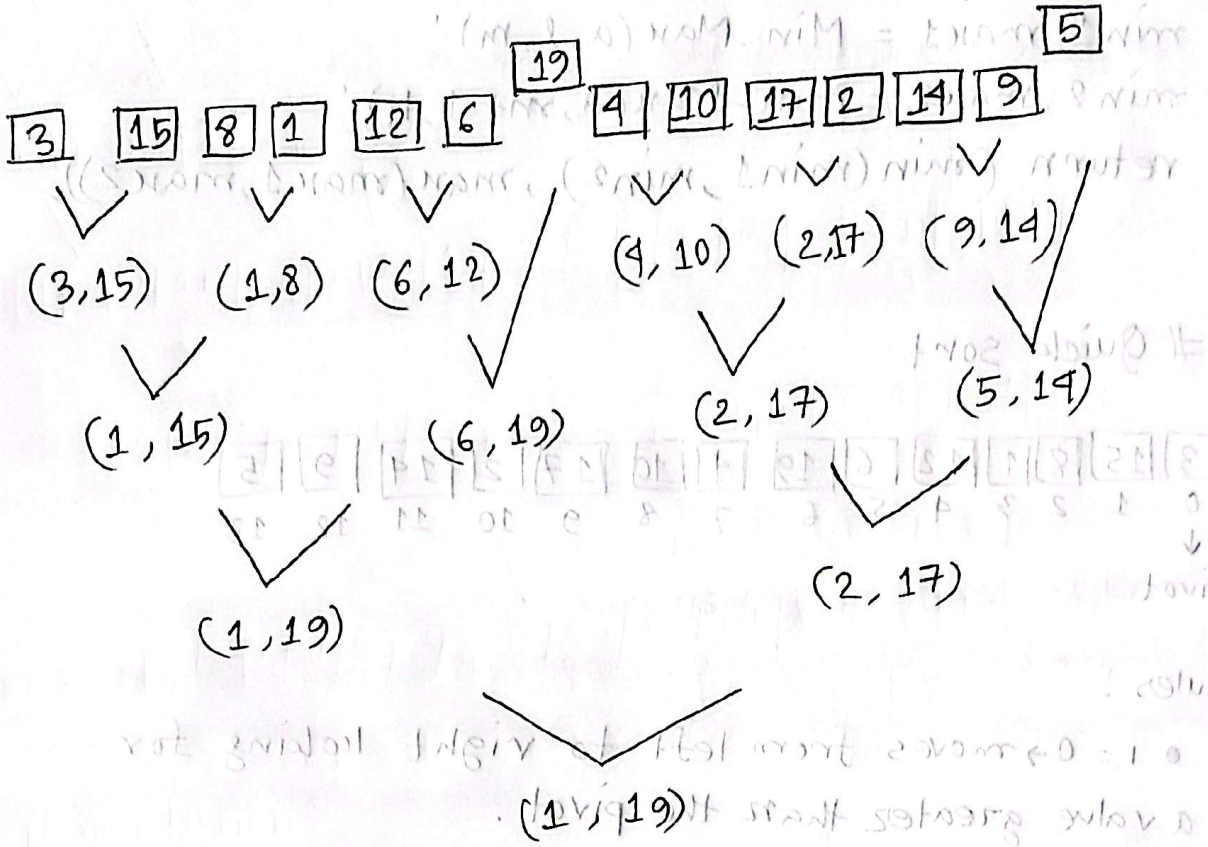
```
while (i<=m && j<=h){
    if(a[i]<=a[j])
        temp[k++] = a[i++];
    else
        temp[k++] = a[j++];
}
if (i>m){
    while (j<=h){
        temp[k++] = a[j++];
    }break;
}
else{
    while (i<=m){
        temp[k++] = a[i++];
    }break;
}
for(p=0; p<k; p++){
    a[l+p] = temp[p];
}
}
```

Space complexity = $\log n + 2n \approx O(n)$

- $\log n$ → height of tree
- $2n$ → main array and temp array

# Find MinMax (Divide and Conquer Approach)

* note: no coquer step needed *




Implementation:

```
Min_Max (a[], l, h) {
    if (l==h)
        return (a[l], a[l]);
    else if (l+1==h) {
        if (a[l]<=a[h]) {
            return (a[l], a[h]);
        }
        else {
            return (a[h], a[l]);
        }
    }

    else {
        m=(l+h)/2;
```

~~min1, min2~~

```
min1, max1 = Min_Max(a, l, m);
min2, max2 = Min_Max(a, m+1, h);
return (min(min1, min2), max(max1, max2));
```

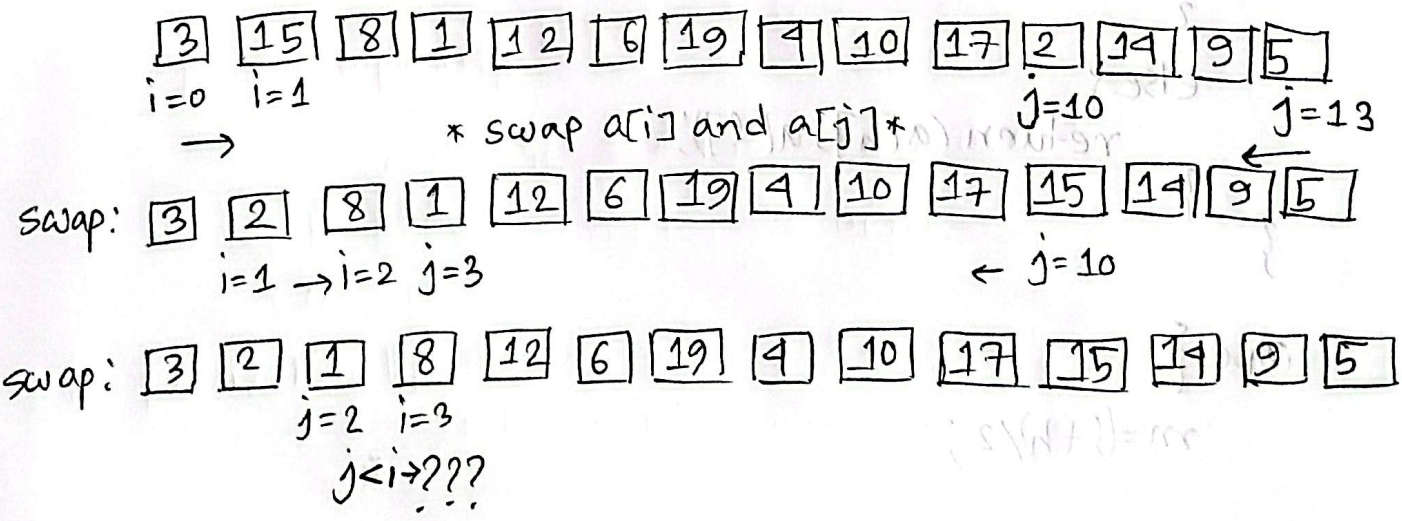
# Quick sort

| 3 | 15 | 8 | 1 | 12 | 6 | 19 | 4 | 10 | 17 | 2 | 14 | 9 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |

↓
Pivot

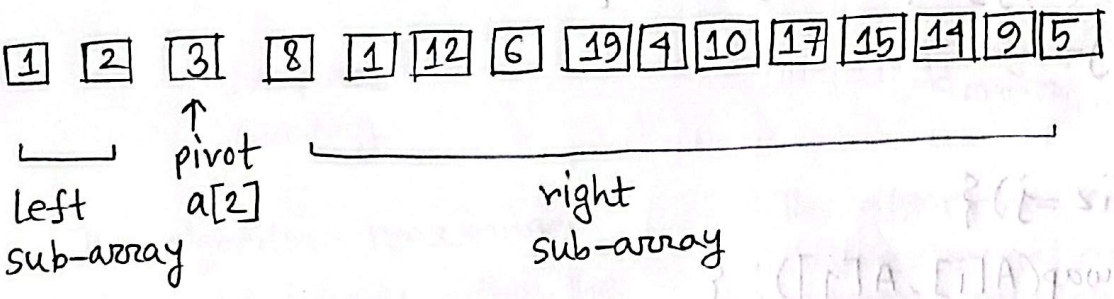
Rules:

- i = 0 → moves from left to right looking for a value greater than the pivot.
- j = n-1 → moves from right to left looking for a value smaller or equal to the pivot.
- Terminator: An infinity (∞) boundary marker is placed at the end of the array to act as a ~~safegaur~~ safeguard constraint so pointer i doesn't overshoot the memory limit.



The crossing point and final pivot placement:

When i>j, the searching loop terminates, ~~and~~ pivot and a[j] are swapped.




Quick sort will now run recursively on those seperate sub-arrays. For left sub-array, pivot → a[0] and ~~right~~ for right sub-array, pivot → a[3], meaning we are choosing the first element as pivot.

Implementation:

```
QS(A,l,h){
   if (l<h){
      p=partition(A,l,h);
      QS(A,l,p-1);
      QS(A,p+1,h);
   }
}

partition (A,l,h){
   pivot = A[l];
   i=l;
   j=h;
```

```
while (i<=j){
    while(A[i]<=pivot){
        i++;
    }
    while(A[j]>pivot && i<h){
        j--;
    }
    if (i<=j){
      swap(A[i], A[j]); }
    else{
      swap (A[j], A[l]); //A[l] is the original pivot position
    }
} re
return j;
}
```

Time complexity : → n elements, → n layers

Worst-case → $O(n^2)$ → happens when reverse sorted

Best-case → $O(n)$

Space complexity :

Worst-case → $n+n = 2n \approx O(n)$

Best-case → $n+\log n \approx O(n)$ (Avg)

**Optimization tip: Picking pivot randomly or using the midpoint helps avoid worst case scenerio.

# Sorting Classification

## In-place Sorting

The algorithm rearranges elements directly within the original array space.

Example: Quick sort, Bubble sort.

## Out-of-place Sorting

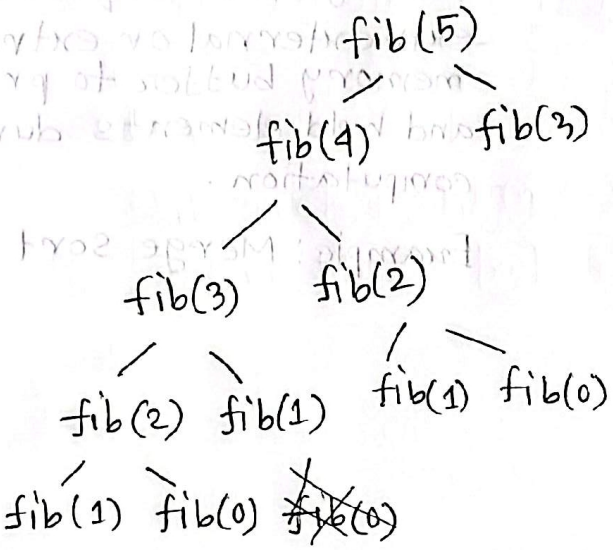
The algorithm requires an ~~extra~~ external or extra memory buffer to process and hold elements during computation.

Example: Merge sort.

# Dynamic Programming

It introduces how to solve problems by combining solutions to subproblems. ~~, illustrated using a recursion tree for calculating~~

#Fibonacci :




We don't repeat the already done calculations or works, which saves us time and makes the solution more efficient.

Core properties of DP problems:

* Optimal substructure : The optimal solution to the overall problem can be constructed efficiently from the optimal solutions of its subproblems.

* Overlapping subproblems : The recursive evaluation chain repeatedly visits the exact same subproblems over and over again rather than always generating brand-new unique ones.

CamScanner

### Optimization technique: If there are common/repeated things, we don't need to recal-culate them.

Approaches:

- Top-down: Moves from the main problem down to the base cases (typically implemented via recursion with memoization).
- Bottom-up: Starts solving from the smallest base cases and builds up to the target answer (typically implemented via iterative loops using arrays).

#Bottom-up approach for Fibonacci:

```
int fib(int n){
    for(
    f[0] = 0;
    f[1] = 1;
    for (i = 2; i<=n; i++)
        fib[i] = fib[i-1] + fib[i-2];
    return f[n];
}
```

We are storing the values in an array (f [n+1]).

# Top-down approach for Fibonacci:

```
int fib(n){
    if(n==1||n==0){
        return n;
    }
    else if(fib_arr[n]!=NULL){
        return fib_arr[n];
    }
    else{
        fib_arr[n] = fib(n-1)+fib(n-2);
        return fib_arr[n];
    }
}
```

Time complexity : (without DP)

| Base case boundary:
| $n-k=0$
| $\Rightarrow k=n$

$$T(n) = T(n-1)+T(n-2)+c$$
$$\approx 2T(n-1)+c \quad [T(n-1)\approx T(n-2)]$$
$$= 2\{2T(n-2)+c\}+c$$
$$= 2^2T(n-2)+2c+c$$
$$= 2^3T(n-3)+2^2c+2^1c+2^0c$$
$$= \ldots\ldots$$
$$= 2^kT(n-k)+(2^{k-1}+\ldots\ldots 2^2+2^1+2^0)c$$
$$= 2^n.1+(2^n-1)c$$
$$\approx O(2^n) \quad [\text{With DP} \rightarrow O(n)]$$

- Top-down → Memoization
  Using a recursive strategy that remembers past results in a lookup array or table.
- Bottom-up → Tabulation
  Using an iterative loop strategy to systematically fill out a table from smallest subproblems to largest.

# Zero-one Knapsack (DP → Bottom-up approach)

Core Rule: Either take full or leave it (no fraction)

~~Goal:~~

Capacity = 8

| Profit | 3 | 5 | 4 | 7 | 2 |
|---|---|---|---|---|---|
| Weight | 2 | 3 | 4 | 5 | 6 |

Solution:

| W | P | ID \ C | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 → Capacity |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 3 | 1 | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 3 | 3 → Profit |
| 3 | 5 | 2 | 0 | 0 | 3 | 5 | 5 | 8 | 8 | 8 | 8 |
| 4 | 4 | 3 | 0 | 0 | 3 | 5 | 5 | 8 | 8 | 9 | 9 |
| 5 | 7 | 4 | 0 | 0 | 3 | 5 | 5 | 8 | 8 | 10 | 12 |
| 6 | 2 | 5 | 0 | 0 | 3 | 5 | 5 | 8 | 8 | 10 | 12 → Final Result |

Here, every cell represents the subproblems.

CamScanner

Whichever cell we are currently evaluating, all rows above it must be taken into consideration. We must consider the current row first, and then the previous ones. We store the best possible solution in the cell for that subproblem.

Conditions:

When you can't take current weight, ~~(wi ≤ j)~~

$$d[i][j] = d[i-1][j]$$

[ the max capacity of the sub-knapsack you are looking at (the column) ]

When you can take the weight,

$$d[i][j] = \max(d[i-1][j], Profit_i + d[i-1][(j) - w_i])$$

Note: Capacity থেকে current weight বিয়োগ করে যে value পাই, সে column এ গিয়ে immediate profit যোগ করব।

Backtracking to find selected items:

* Start from the very last cell (final result).
* Move upwards through the rows as long as the values remain same.
* The moment the value changes (is not same as the row above), it means object was selected.
* Repeat from that new cell's perspective.

CamScanner

# Coin change problem:

Bill to pay →

| Coins ↓ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 (B=9) |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 2 | 0 | 1 | 1 | 2 | 2 | 3 | 3 | 4 | 4 | 5 |
| ✓ 4 | 0 | 1 | 1 | 2 | 1 | 2 | 2 | 3 | 2 | 3 |
| 5 | 0 | 1 | 1 | 2 | 1 | 1 | 2 | 2 | 2 | 2 |
| 7 | 0 | 1 | 1 | 2 | 1 | 1 | 2 | 1 | 2 | (2) |

[4-4] [9-5]

↳ Final result

if coin is not chosen,

$$d[i][j] = d[i-1][j]$$

if coin is chosen,

$$d[i][j] = \min(d[i-1][j], 1 + d[i][j - c_i])$$

$c_i$: ith coin value

# Longest common subsequence: (not necessarily consecutive, should be in same ~~serial~~ order)

LCS

| | 0 | E ✓ | N | G | I | N ✓ | E | E ✓ | R ✓ | I ✓ | N ✓ | G ✓ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| G | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| E | 0 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 |
| N | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 3 |
| E | 0 | 1 | 2 | 2 | 2 | 2 | 3 | 3 | 3 | 3 | 3 | 3 |
| R | 0 | 1 | 2 | 2 | 2 | 2 | 3 | 3 | 4 | 4 | 4 | 4 |
| A | 0 | 1 | 2 | 2 | 2 | 2 | 3 | 3 | 4 | 4 | 4 | 4 |
| T | 0 | 1 | 2 | 2 | 2 | 2 | 3 | 3 | 4 | 4 | 4 | 4 |
| I | 0 | 1 | 2 | 2 | 3 | 3 | 3 | 3 | 4 | 5 | 5 | 5 |
| N | 0 | 1 | 2 | 2 | 3 | 4 | 4 | 4 | 4 | 5 | 6 | 6 |
| G | 0 | 1 | 2 | 3 | 3 | 4 | 4 | 4 | 4 | 5 | 6 | 7 |

↳ Final Result

If matches: $d[i][j] = d[i-1][j-1] + 1$

If not matches: $d[i][j] = \max(d[i-1][j], d[i][j-1])$

LCS ~~Backtrack~~:

1. If ~~the~~ characters of ~~current~~ row and ~~column a~~ match, move ~~diagonally up left~~ ($d[i-1][j-1]$) ~~and~~ record that character

Soln: Enering (considered left direction when values were equal)

Application: Human gene analysis.

## Lcs Backtracking Steps:

1. Start from the bottom-right corner.
2. If characters match → add that character to the answer and move diagonally.
3. If character don't match → we ~~check~~ compare d[i-1][j] and d[i][j-1] for the maximum value and move towards that direction. If values are equal, either direction works.
4. We keep repeating the process.

# For multiple solutions: Explore both directions when values are equal.

# SOS (Sum over Subsets)

This efficiently computes values for all subsets of a set.

Example:

Array = {3, 2, 6, 8, 5}
1 2 3 4 5

[d[i][j] → Is it possible to make sum j using the first i elements? (boolean)

Target sum = 10

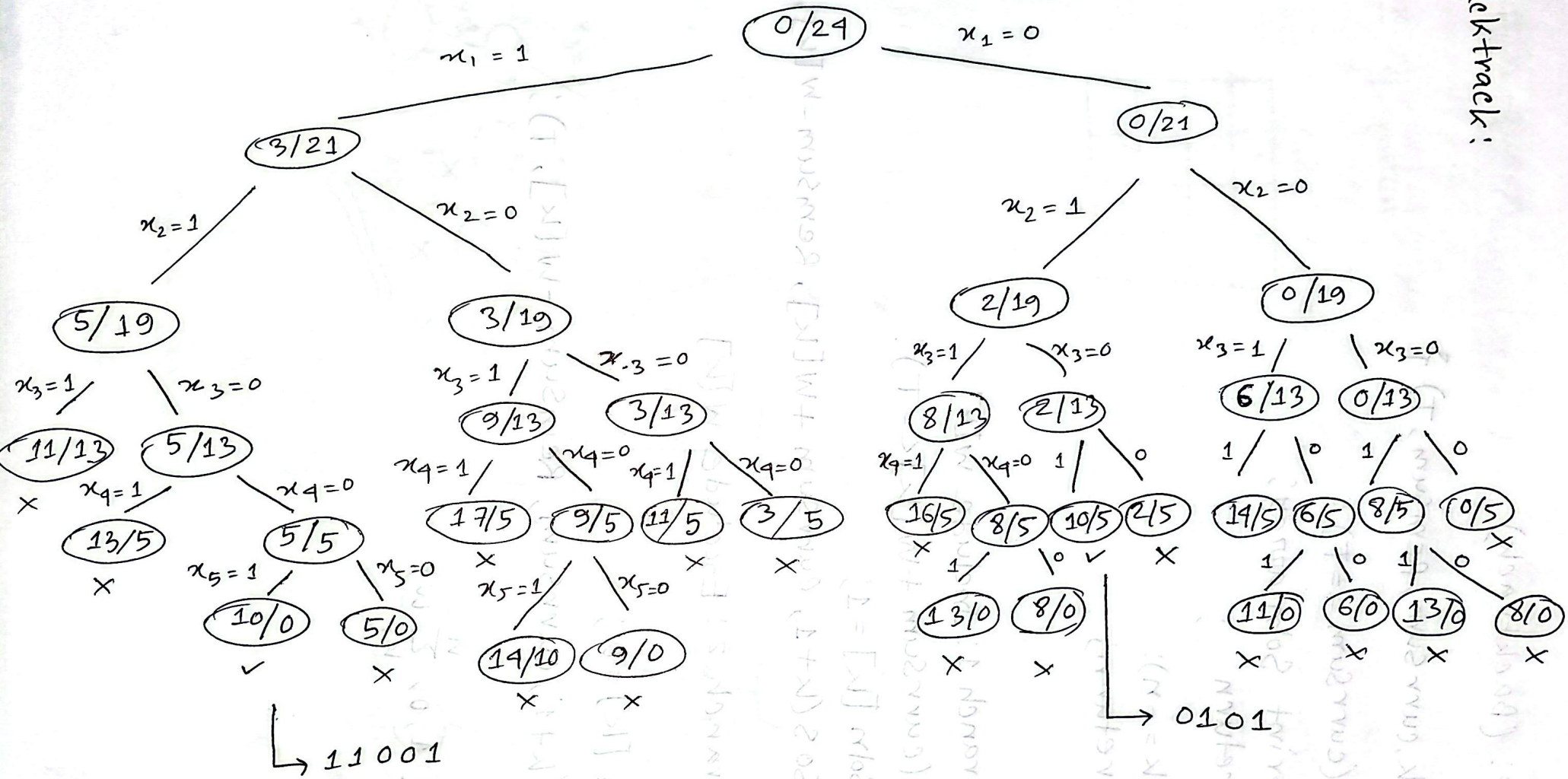
j → target sums from 0 to 10

| | X | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Base case (No elements) ← | Ntg | T | F | F | F | F | F | F | F | F | F | F |
| i = 0 | 3 | T | F | F | T | F | F | F | F | F | F | F |
| 1 | 2 | T | F | T | T | F | T | F | F | F | F | F |
| 2 | 6 | T | F | T | T | F | T | T | F | T | T | F |
| 3 | 8 | T | F | T | T | F | T | T | T | T | T | T |
| 4 | 5 | T | F | T | T | F | T | T | T | T | T | T |

Steps: 1. d[i][0] = T (sum 0 is possible with no elements)
d[0][j] = F (Positive sum is not possible with 0 elements)

2. Choice : Take it or don't take it. (for d[i][j])

Backtrack:




$(T.S - C.S) <= A.\forall S$

- Target Sum
- Current Sum
- Available Sum (Remaining)

Complexity = $O(b^d \times c)$

- $d$ → depth of tree → $n$
- $c$ → cost per node → $O(1)$
- $b$ → branching factor → 2

Worst case → $O(2^n)$

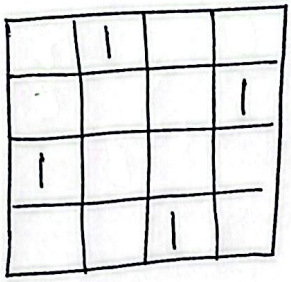
Pseudocode: (Backtrack)

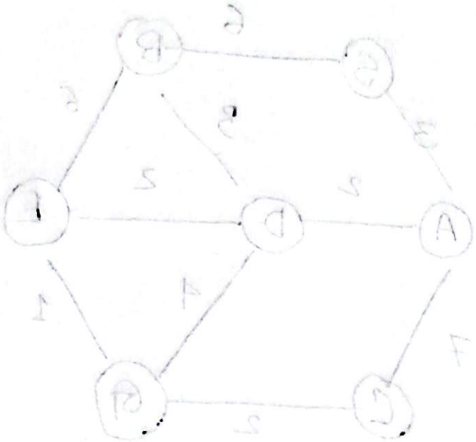
```
SOS(k, currSum, RemSum, T) {
    if (currSum == T):
        print Soln array;
        return ;
    if(k == n):
        return;

    ##
    // Branch 1: Include w[k]
    if (currSum + W[k] <= T):
        Soln [k] = 1;
        SOS(k+1, currSum + W[k], RemSum - W[k], T);

    // Branch 2: Exclude W[k]
    ##if
    soln [k] = 0;
    SOS(k+1, currSum, RemSum - W[k], T);
```

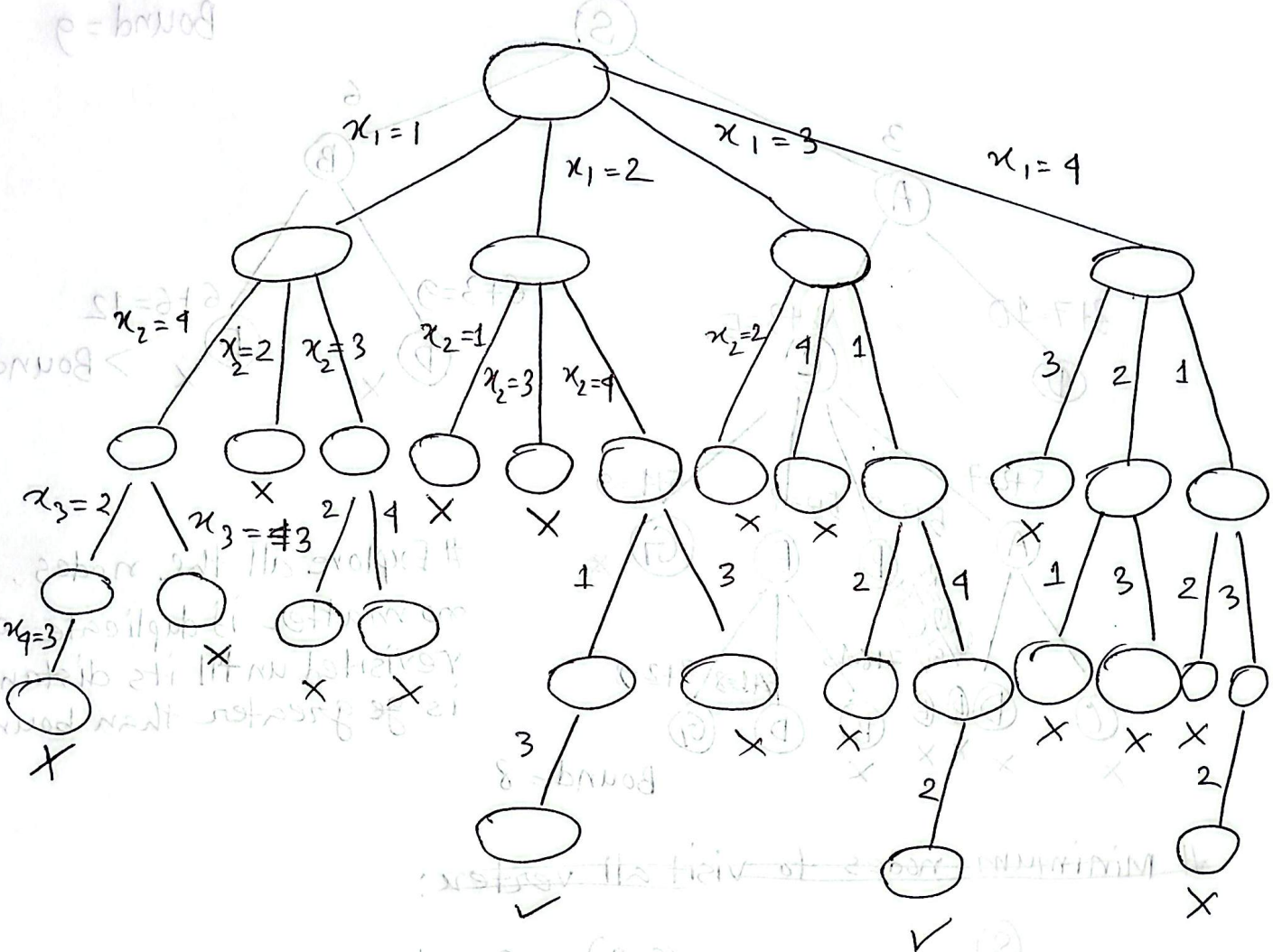
Call: $SOS(1, 0, \sum_{k=1}^{n} w)$

# n-queen

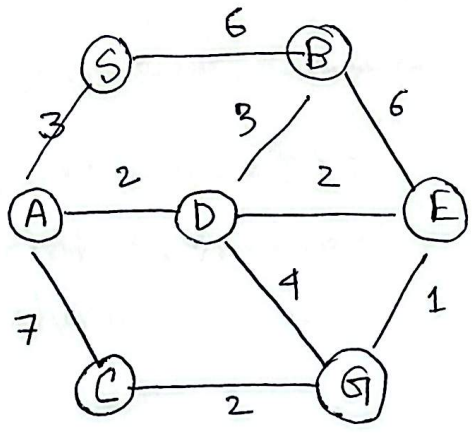
Ex: 4-queen



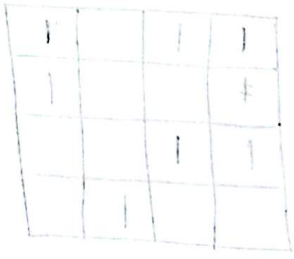


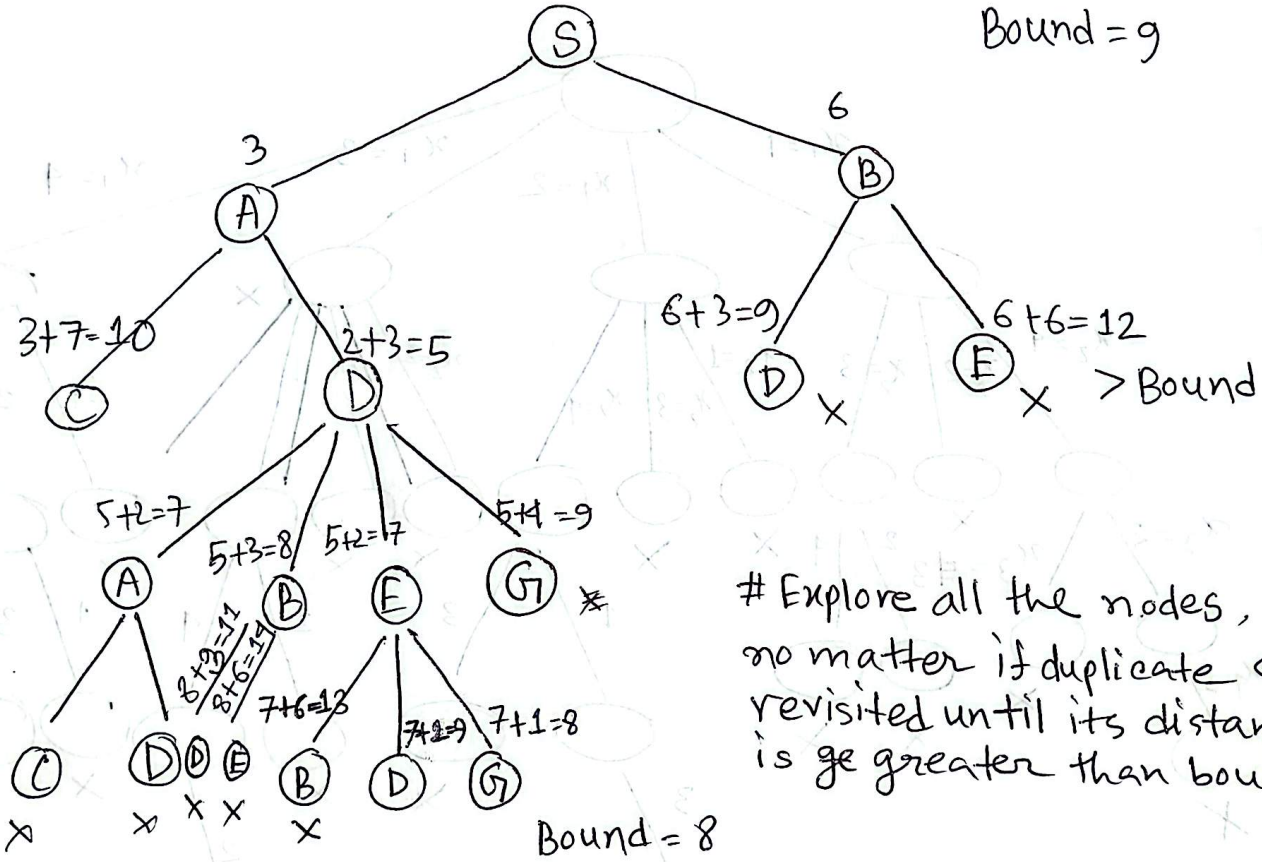


# Branch & Bound (BFS)



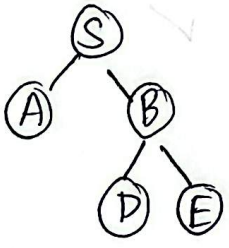

Goal: Find shortest path from S to G.






# Explore all the nodes, no matter if duplicate or revisited until its distance is ge greater than bound.

~~# Minimum nodes to visit all vertex~~:




(S, B) → Exact answer

S, D, E → Approximate answer.